

Documentation that Engineers Actually Use

Ship **repeatable, auditable** AI infrastructure with docs-as-code that are owned and versioned.

Outcome: one repo, one truth



Why Documentation Matters

Faster Onboarding

Reduce "how do I...?" pings and get new team members productive faster

Reduced Deployment Risk

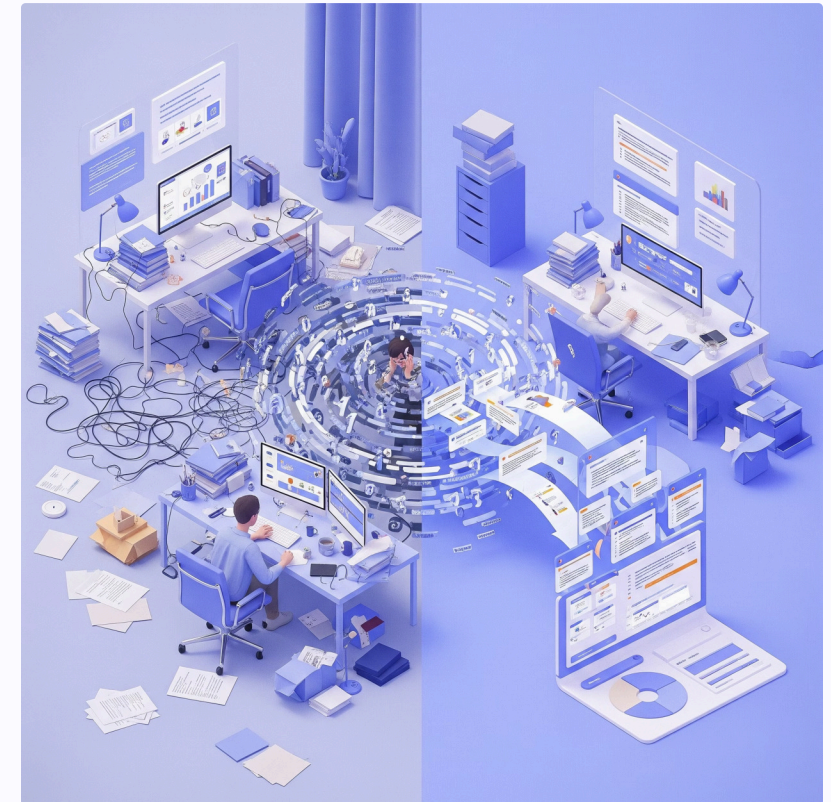
Repeatable runbooks ensure consistent operations across environments

Audit Enablement

Clear evidence for security, compliance, and cost reviews

Capstone Success

Provides clear artifacts for grading and stakeholder presentations



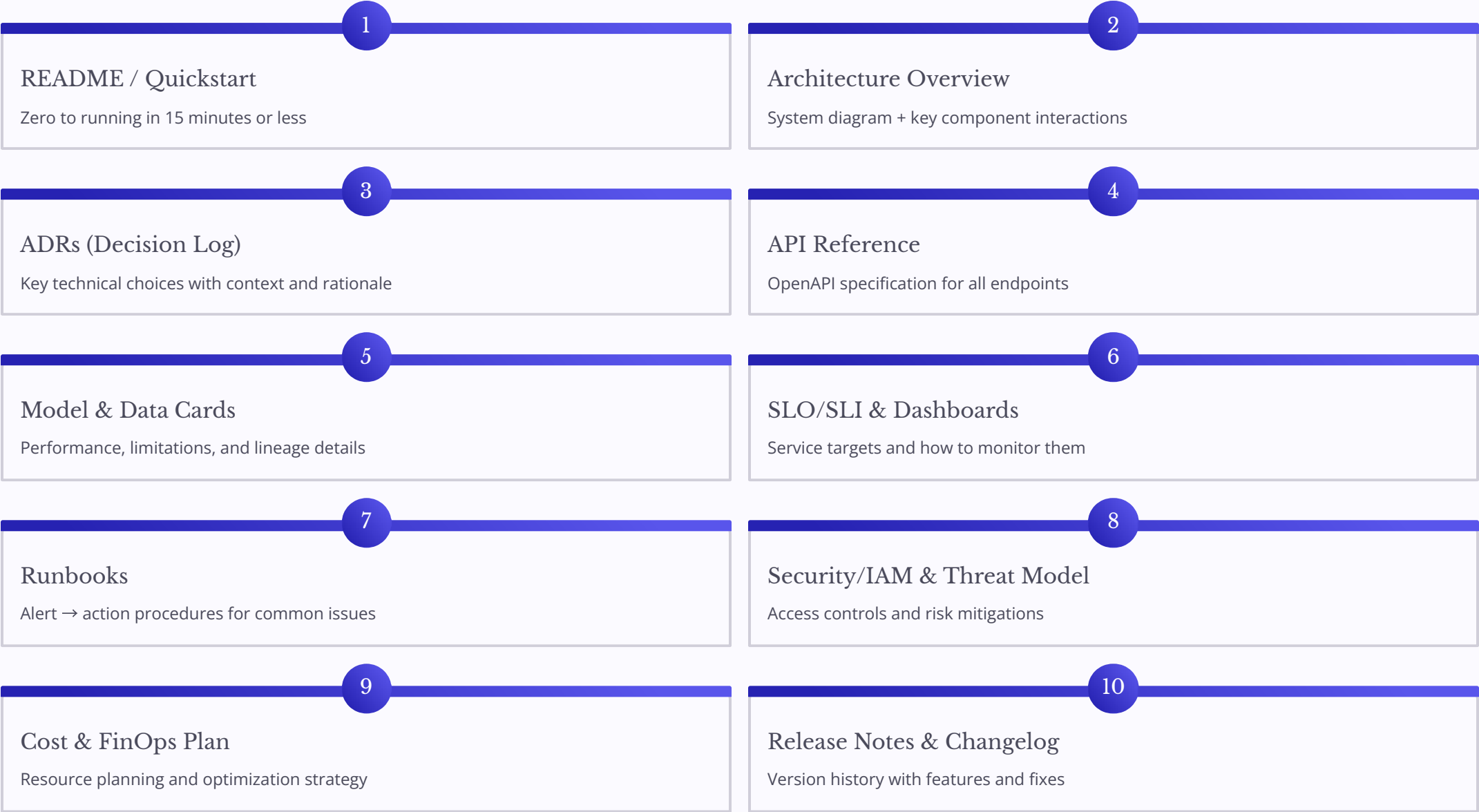
Audience Needs Drive Documentation Types

Audience	Their Needs	Documentation Artifacts
Engineers & ML Teams	How to build and run	README, Quickstart, ADRs, API Reference, Model/Data Cards
SRE/Ops	How to keep up and running	Runbooks, SLOs, Dashboards Guide, HA/Failover Procedures
Security & Compliance	Evidence & controls	IAM Matrix, Audit Log Specifications, Data Handling Procedures
Product & Executives	ROI & risk assessment	KPIs, Cost Plan, Release Notes

Documentation must be tailored to each audience's specific information needs while maintaining a single source of truth.

Core Documentation Set

The minimum viable documentation package for your Capstone project:



Repository Structure (Docs-as-Code)

```
repo/
├── README.md
├── docs/
│   ├── architecture.md
│   ├── diagrams/ (Mermaid/PlantUML svgs)
│   ├── adr/ADR-0001-model-serving.md
│   ├── api/openapi.yaml
│   ├── model_card.md
│   ├── data_card.md
│   ├── slo_sli.md
│   ├── dashboards_guide.md
│   ├── runbooks/
│   │   ├── latency.md
│   │   ├── drift.md
│   │   └── outage-drill.md
│   ├── security_iam.md
│   ├── cost_finops.md
│   ├── release_notes.md
│   └── contributing.md
└── mkdocs.yml (or docusaurus.config.js)
```

Key Benefits

- Documentation lives **alongside code**
- Changes to both are **reviewed in the same PR**
- Generated site provides **clean UI** for consumption
- History tracked through **git versioning**
- CI/CD can **validate and test** documentation
- Developers use **familiar tools** (editors, git)



Writing Principles for Technical Documentation



Docs-as-code

Version-controlled, reviewed via PRs, and treated with the same rigor as production code



Single source of truth

OpenAPI specs, IaC templates, and dashboard configs should be checked in and generated from source



Templatize repeatable content

Use consistent skeletons for runbooks, model cards, and other structured docs



Clear ownership

Every page needs an **OWNER** field and last updated date for accountability



Make it runnable

Include tested, copy-paste-ready commands that actually work

Architecture Overview Template

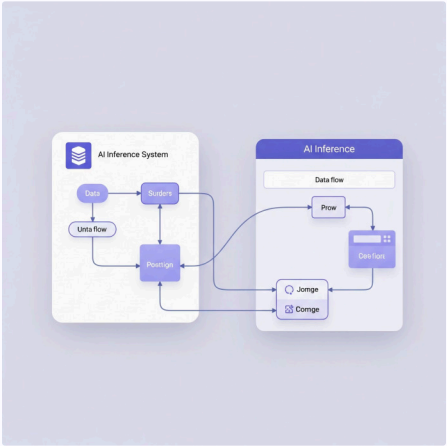
Essential Components

- **Purpose:** Explain end-to-end flow & key tradeoffs
- **Include:** Context, constraints, choices, risks, and operational considerations
- **Format:** Mermaid/PlantUML for version-controlled diagrams
- **Detail level:** Component interactions, not implementation details

Tip: Update diagram after every architecture change

flowchart LR

```
A[Producers] --> |events| K[Kafka]
K --> B[Batch ETL (Airflow)]
K --> S[Stream (Flink)]
B --> BR[Bronze S3] --> SI[Silver Parquet] --> GO[Gold Features]
GO --> T[Training (DDP/DeepSpeed)]
T --> R[Model Registry]
R --> D[(Docker Image)]
D --> K8S[Inference on K8s]
K8S --> FE[Feature Store (online)]
K8S --> G[Grafana/Prometheus]
```

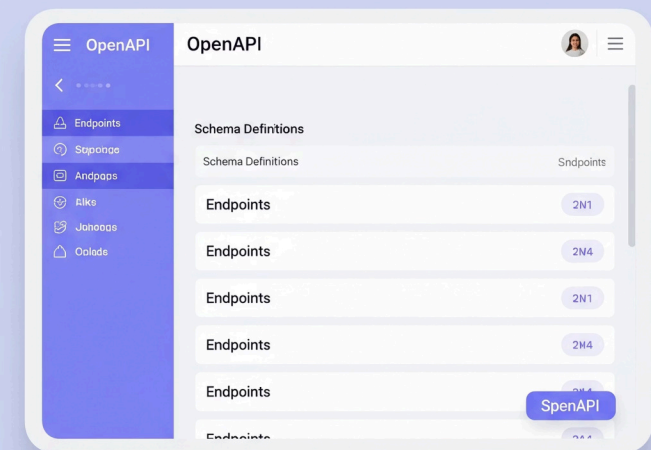


API Reference (OpenAPI)

```
openapi: 3.0.3
info:
  title: Capstone Inference API
  version: 1.0.0
paths:
  /healthz:
    get:
      responses:
        "200": {description: ok}
  /predict:
    post:
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Request' }
      responses:
        "200":
          description: success
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Response' }
components:
  schemas:
    Request:
      type: object
      properties: { text: {type: string} }
      required: [text]
    Response:
      type: object
      properties:
        model: {type: string}
        label: {type: string}
        score: {type: number, format: float}
```

Best Practices

- **Single source:** OpenAPI spec should be used to generate both docs and code
- **Validate:** Test spec matches actual API behavior
- **Publish:** Export static HTML with ReDoc or Scalar
- **Examples:** Include request/response examples



Model Card Template

Model Card — **Owner:** @team-ml • **Version:** v1.2.0 • **Date:** 2025-08-28

Use cases: sentiment on short texts (≤ 256 tokens)

Training data: corpus v2025-06 (curated, de-identified)

Metrics (eval): F1=0.89, Acc=0.91 (dataset hash: abc123)

Latency (p95): 240 ms @ 200 rps on A10G

Limitations: poor for dialectal slang; OOD long docs

Safety notes: output filter v0.7 enabled

Update policy: retrain monthly; drift $\text{PSI} > 0.25$ triggers review

- ❏ Model cards provide essential information about AI models for both technical and non-technical stakeholders. They document performance characteristics, limitations, and operational parameters in a standardized format.

Data Card Template

Data Card — Corpus v2025-06

****Owner:**** @data-eng

****Source:**** user feedback + public corpora

****Collection period:**** 2025-04 → 2025-06

****License:**** internal use, no redistribution

****PII handling:**** de-ID pipeline v2; tokenization; approvals logged

****Schema:**** {id, text, label, ts, region}

****Quality checks:**** null<0.1%, dup<0.2%, lang=en>98%

****Bias review:**** region distribution balanced $\pm 5\%$ vs target

****Retention:**** raw 90d (cold), features 365d

****Lineage:**** DVC commit 9f0d...; GE checkpoint ge://capstone/daily

Why Data Cards Matter

Data cards provide critical context about training and evaluation datasets that:

- Document **data provenance** and lineage
- Highlight **potential biases** or limitations
- Clarify **privacy considerations** and handling procedures
- Enable **reproducibility** of results
- Support **compliance** with regulations



SLO/SLI Specification Template

SLOs & SLIs

****Latency SLO:**** p95 < 300ms (monthly)

****Availability SLO:**** 99.9% (monthly)

****Error SLO:**** 5xx < 0.1% (rolling 1h)

****Drift SLO:**** PSI(user_stats.ctr_30d) < 0.25 (daily)

****SLI Queries (PromQL examples):****

- p95: `histogram\_quantile(0.95, sum by (le)(rate(http\_request\_duration\_seconds\_bucket{job="infer"}[5m])))`

- avail: `1 - (sum(rate(http\_requests\_total{status=~"5.."}[5m])) / sum(rate(http\_requests\_total[5m])))`

Key Components

- **SLO:** Target level of service
- **SLI:** Actual measured performance
- **Timeframe:** Measurement period
- **Query:** How to calculate the metric



Runbook Template: High Latency Response

Runbook — High p95 Latency

****Alert:**** p95 > 300ms for 10m (severity: page)

****Owners:**** @platform-oncall • ****Dashboards:**** /grafana/api

Diagnose

1. Check error rate panel (5xx spike?).
2. Compare traffic vs replicas (HPA scaling?).
3. Inspect feature store p95; if high → cache hot keys.

Mitigate

- Scale up: `kubectl scale deploy infer -n ml --replicas=+2`
- Temporarily increase HPA max to 30.
- If model vNext just deployed → rollback via registry.

Root Cause

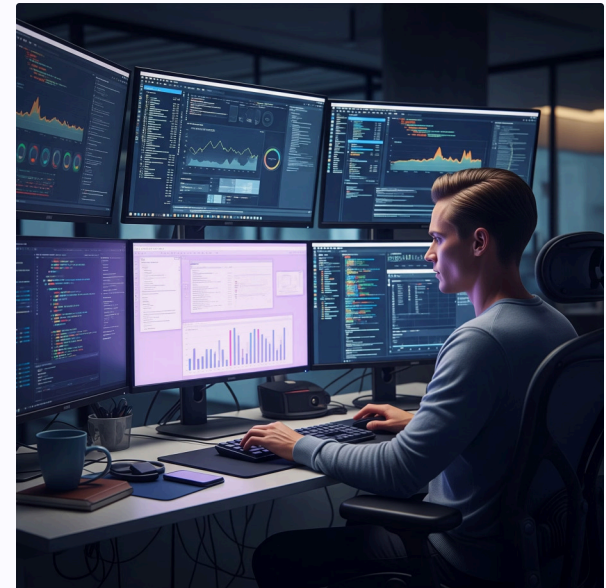
Capture trace IDs & graphs, open incident ticket with timelines.

Prevent

Tune batch size, enable KV cache, bump pod CPU limits.

Effective Runbooks

- **Actionable:** Clear, copy-paste commands
- **Contextual:** Link to dashboards and alerts
- **Comprehensive:** Cover diagnosis, mitigation, and prevention
- **Tested:** Validated through drills



Security, IAM & Threat Model

Authentication Paths

- OAuth2/OIDC for edge services
- mTLS for service mesh communications
- Key rotation schedule and procedures

RBAC Matrix (Role-Based Access)

Role	Action	Scope	Method
deployer	deploy	Helm prod namespace	Argo CD app
service	read features	online store	IAM condition: tenant=foo

STRIDE Threat Model

Documented threats and mitigations for:

- Spoofing identity
- Tampering with data
- Repudiation of actions
- Information disclosure
- Denial of service
- Elevation of privilege

Evidence Collection

- Signed container images
- KMS encryption policies
- Network security policies
- Audit log configurations

Cost & FinOps Documentation

Budget Tracking

- Monthly cap: \$X,XXX
- GPU hours: planned vs. actual
- Cost per 1K requests target: \$X.XX
- Current trend visualization

Storage Optimization

- Lifecycle rules by data tier
- Retention periods by data importance
- Compression strategies
- Hot/cold storage transitions

Resource Controls

- Auto-shutdown for idle resources
- Quota limits by environment
- Right-sizing recommendations
- Spot instance usage policy

Monitoring & Tools

Link to specific dashboards for:

- Kubecost allocation views
- Cloud billing exports
- Resource utilization metrics
- Cost anomaly detection



Pro tip: Document cost reduction initiatives and their measured impact to demonstrate engineering value.

Dashboards Guide: Reading Your Metrics

Dashboard Structure

- SLO tiles at the top
- Red/amber/green thresholds
- Owner listed prominently
- Time range controls

Navigation Features

- Panel → logs links
- Panel → traces links
- Release markers
- Alerting thresholds

Responsibility Matrix

- Who to page for each board
- Expected response times
- Escalation paths
- Incident classification

Effective dashboard documentation explains not just what metrics mean, but **how to act on them** when they indicate problems.



Developer Quickstart

```
# Quickstart
```

```
## 1) Setup
```

```
- `pyenv local 3.10 && pip install -r requirements.txt`  
- `docker build -t ghcr.io/org/infer:$USER-dev`
```

```
## 2) Run locally
```

```
- `docker run -p 8080:8080 ghcr.io/org/infer:$USER-dev`  
- `curl :8080/healthz`
```

```
## 3) Deploy to dev (ephemeral)
```

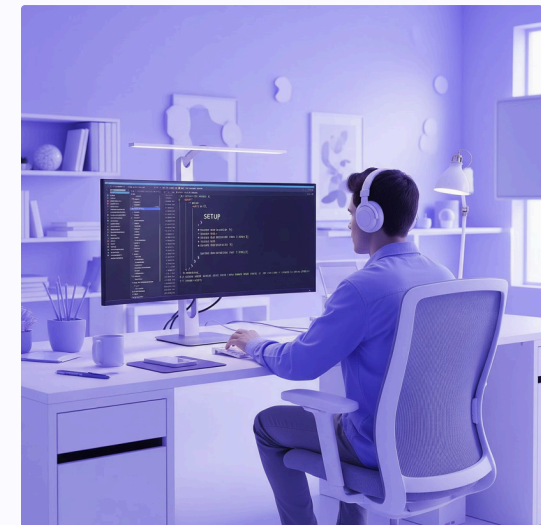
```
- `make ns=dev-$USER deploy`  
- `make smoke BASE=https://infer-dev-$USER.local`
```

```
## 4) View dashboards
```

```
- `make port-forward-grafana`
```

Quickstart Best Practices

- **Time-boxed:** Should take ≤ 15 minutes
- **Self-contained:** Minimal external dependencies
- **Verifiable:** Clear success indicators
- **Environment-aware:** Works on standard dev machines



Architecture Decision Records (ADRs)



ADR-0001 — Triton over FastAPI-only for Inference

****Context:**** Multi-model GPU serving, need dynamic batching.

****Decision:**** Use NVIDIA Triton + gRPC; wrap with HTTP gateway.

****Status:**** Accepted (2025-08-20).

****Consequences:**** Higher ops complexity; better throughput; future multi-model support.

ADR Benefits

- Documents decision context for future teams
- Forces structured thinking about alternatives
- Captures trade-offs explicitly
- Creates history of technical evolution

Best Practices

- Keep ADRs small, one per decision
- Link alternatives considered
- Include performance data when relevant
- Update status if decisions change

Release Notes & Changelog

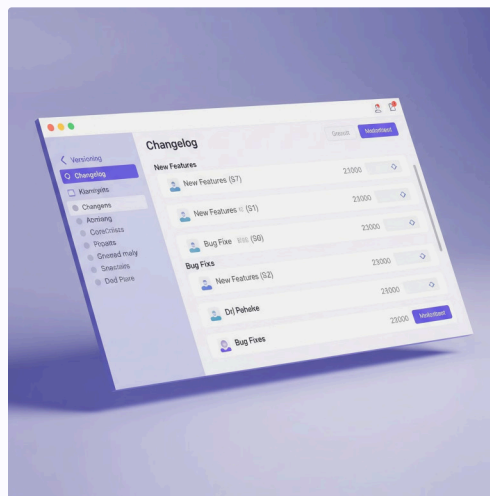
Structure

- Follow **SemVer** versioning (MAJOR.MINOR.PATCH)
- Include for each release:
 - New features & improvements
 - Bug fixes with issue references
 - Performance improvements
 - Risk notes and special precautions
- Attach supporting artifacts:
 - Performance test reports
 - Grafana screenshots
 - Cost delta analysis

Example Entry

v1.3.0 — 2025-08-26

- + Distilled model v1.3 (−18% latency, −12% \$/1k)
- + Drift detector thresholds tuned
- Fix: feature store timeout handling
- Risk: new batch size; canary 10% for 2h



Documentation Automation

CI/CD Pipeline Components

- Build docs site on every merge
- Lint prose with **Vale**
- Validate links with **lychee**
- Render Mermaid to SVG
- Validate OpenAPI JSON schema
- Check for outdated content



Example GitHub Action

```
name: docs
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: ruby/setup-ruby@v1
      - run: pip install mkdocs-material mermaid-js
      - run: mkdocs build --strict
      - uses: peaceiris/actions-gh-pages@v3
    with:
      github_token: ${{secrets.GITHUB_TOKEN}}
      publish_dir: ./site
```

Automation ensures documentation stays accurate and up-to-date without manual intervention.

Documentation Quality Checklist

Ownership & Currency

Every page has **clear owner + last updated date**

Developer Onboarding

README bootstraps a new developer in ≤ 15 minutes

Architecture Accuracy

Architecture diagram matches current production deployment

API Documentation

OpenAPI spec matches deployed API (verified in CI)

Model & Data Transparency

Cards include complete metrics, lineage information, and limitations

Operational Readiness

SLOs are measurable with linked PromQL and runbooks tested in drills

Security & Cost

IAM matrix, threat model, cost plan, and dashboards all linked and current

Version History

Release notes for at least the last two versions included

Summary: Documentation Best Practices

Document What You Run

Focus on actual production systems, not aspirational designs or future plans

Keep Docs Near Code

Use docs-as-code approach with automated builds and validation checks

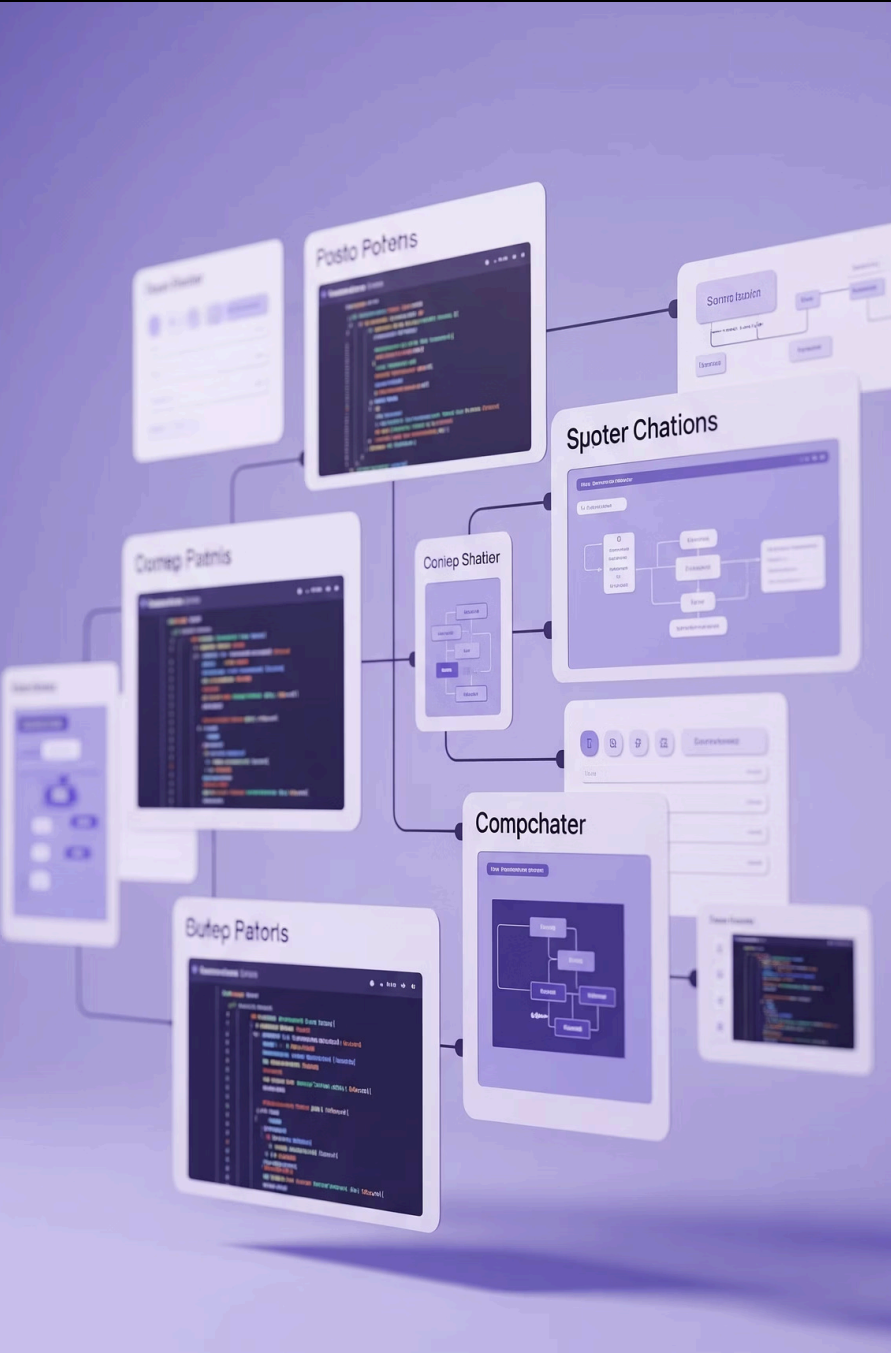
Make It Actionable

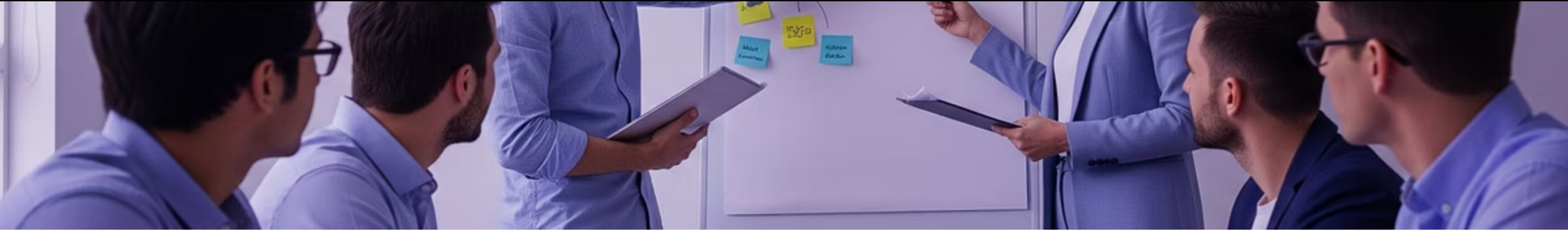
Include specific commands, queries, owner information, and detailed runbooks

Focus on Quality Over Quantity

Good documentation means fewer pages with greater clarity and precision

Remember: Engineers will use documentation that makes their job easier—and ignore what doesn't.





Discussion: Prioritizing Your Documentation

Which doc will you finish first for your Capstone demo?

N

Architecture Overview

Shows the big picture and how components fit together. Demonstrates technical vision and system design skills.



Runbook

Proves operational readiness and attention to reliability. Shows you've thought through failure scenarios.



Model/Data Cards

Demonstrates ML engineering rigor and understanding of model/data lifecycle management.